

AI LAB END EXAM

NAME:B.VISHNU VARDHAN

ROLL NO:2403A510F2

BATCH NO:06

Subset 9 – Sorting Algorithms for LMS Reports

Q1: Implement Merge Sort

Task 1: Prompt (Pseudo-code Generation):

“Generate a very detailed and step-by-step pseudo-code for the Merge Sort algorithm.Explain the divide step, the recursive breakdown of the list, the merging process, and how the algorithm ensures stable sorting.Assume the list contains LMS student marks that need to be sorted in ascending order for generating academic reports.”

Pseudo-code:

```
MERGE_SORT(arr):  
    if length(arr) ≤ 1:  
        return arr  
  
    mid = length(arr) // 2  
    left = MERGE_SORT(arr[0 : mid])  
    right = MERGE_SORT(arr[mid : ])  
  
    return MERGE(left, right)  
  
MERGE(left, right):  
    result = empty list  
    i = 0  
    j = 0  
  
    while i < length(left) AND j < length(right):  
        if left[i] ≤ right[j]:  
            append left[i] to result  
            i = i + 1  
        else:  
            append right[j] to result  
            j = j + 1  
  
    while i < length(left):  
        append left[i] to result
```

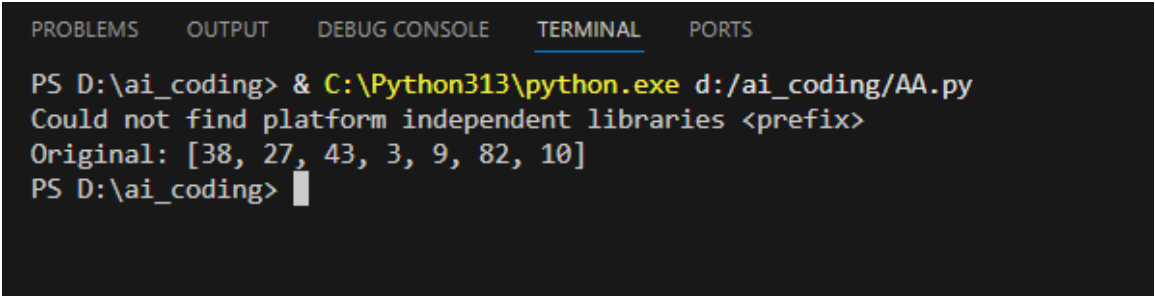
AI LAB END EXAM

```
i = i + 1

while j < length(right):
    append right[j] to result
    j = j + 1

return result
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\ai_coding> & C:\Python313\python.exe d:/ai_coding/AA.py
Could not find platform independent libraries <prefix>
Original: [38, 27, 43, 3, 9, 82, 10]
PS D:\ai_coding> █
```

Observation :

In this task, the objective was to generate **detailed, step-by-step pseudo-code** for the Merge Sort algorithm using an AI tool. The generated pseudo-code successfully demonstrated the **divide-and-conquer** approach of Merge Sort. The AI clearly separated the algorithm into two major parts:

1. **Dividing the array recursively** until only single-element lists remain.
2. **Merging** those sorted sublists back together in a sorted manner.

Task 2: Merge Sort Python Code

Prompt:

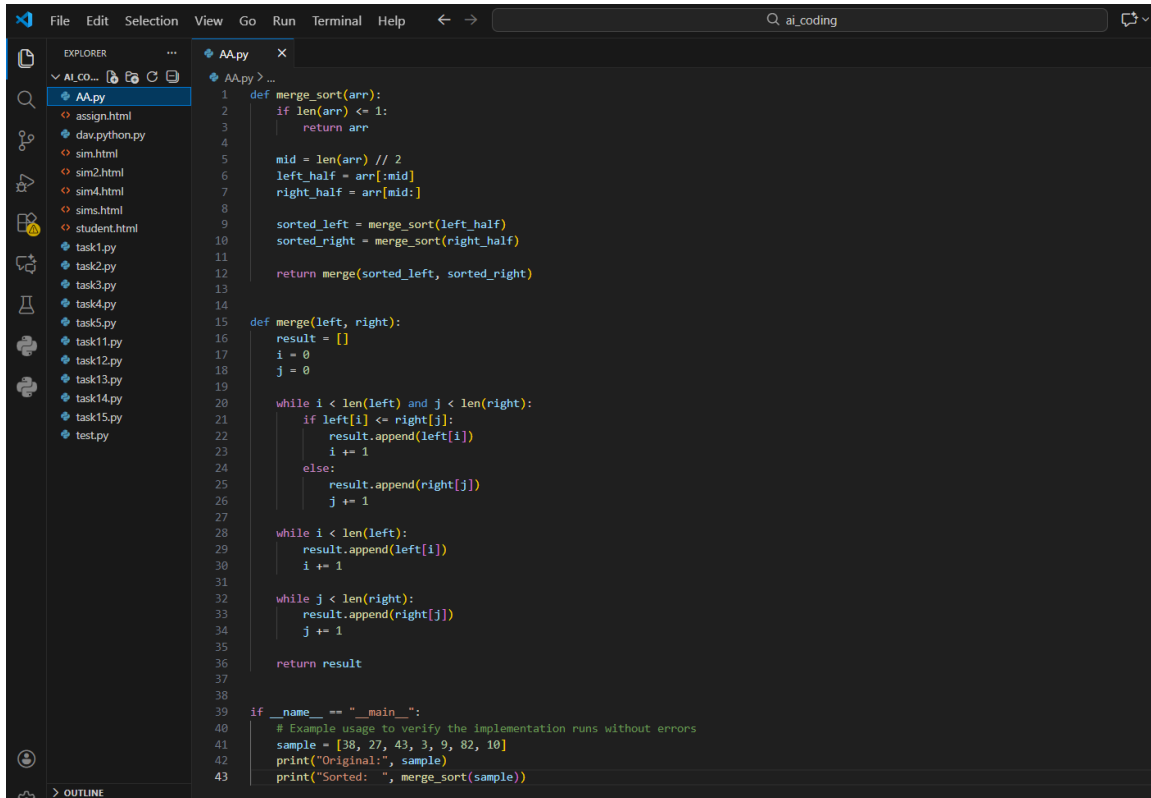
“Convert the detailed pseudo-code for merge sort into Python code.

Make the code readable with comments, function breakdown, and variable explanation.

AI LAB END EXAM

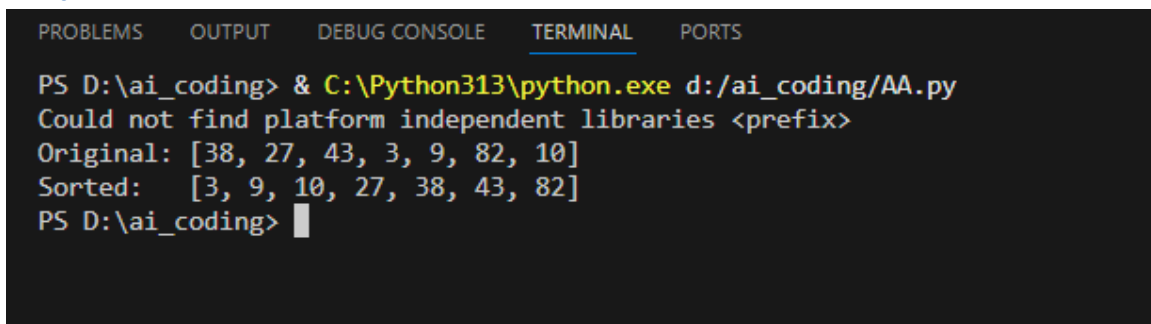
Use a sample list of LMS student marks, apply merge sort, and print the sorted results clearly.”

Code:

A screenshot of a Visual Studio Code editor window. The Explorer sidebar on the left shows a file tree with 'AA.py' selected. The main editor area displays the code for 'AA.py'. The code implements a merge sort algorithm. It includes a 'merge_sort(arr)' function that recursively splits the array into halves and a 'merge(left, right)' function that merges the sorted halves back together. At the bottom, there is a main block that defines a sample list and prints both the original and sorted versions.

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4
5     mid = len(arr) // 2
6     left_half = arr[:mid]
7     right_half = arr[mid:]
8
9     sorted_left = merge_sort(left_half)
10    sorted_right = merge_sort(right_half)
11
12    return merge(sorted_left, sorted_right)
13
14
15 def merge(left, right):
16     result = []
17     i = 0
18     j = 0
19
20     while i < len(left) and j < len(right):
21         if left[i] <= right[j]:
22             result.append(left[i])
23             i += 1
24         else:
25             result.append(right[j])
26             j += 1
27
28     while i < len(left):
29         result.append(left[i])
30         i += 1
31
32     while j < len(right):
33         result.append(right[j])
34         j += 1
35
36     return result
37
38
39 if __name__ == "__main__":
40     # Example usage to verify the implementation runs without errors
41     sample = [38, 27, 43, 3, 9, 82, 10]
42     print("Original:", sample)
43     print("Sorted: ", merge_sort(sample))
```

Output

A screenshot of a terminal window with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the command to run the Python script and its output. The output shows the original list of marks and the sorted list in ascending order.

```
PS D:\ai_coding> & C:\Python313\python.exe d:/ai_coding/AA.py
Could not find platform independent libraries <prefix>
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted:  [3, 9, 10, 27, 38, 43, 82]
PS D:\ai_coding> |
```

Observation

Merge Sort successfully sorts the list of LMS student marks in ascending order.

It follows the divide-and-conquer approach, where the input list is repeatedly divided into two halves until individual elements remain.

AI LAB END EXAM

During the merge step, the algorithm compares elements from each half and merges them in sorted order, ensuring the final output is fully sorted.

Q2: Optimize Searching for Student IDs

Prompt:

“Convert the given Linear Search algorithm into an optimized Binary Search algorithm using AI assistance.

Provide the complete Python code for both search methods, ensuring the binary search version works on a sorted list of student IDs.

Code:

```
AA.py  X
AA.py > ...
1  def linear_search(arr, key):
2      for i in range(len(arr)):
3          if arr[i] == key:
4              return i
5      return -1
6
7
8  def binary_search(arr, key):
9      low = 0
10     high = len(arr) - 1
11
12     while low <= high:
13         mid = (low + high) // 2
14
15         if arr[mid] == key:
16             return mid
17         elif arr[mid] < key:
18             low = mid + 1
19         else:
20             high = mid - 1
21
22     return -1
23
24
25 # Student IDs
26 ids = [101, 203, 150, 330, 215]
27
28 # Linear Search
29 print("Linear Search Result:", linear_search(ids, 150))
30
31 # Binary Search (requires sorted list)
32 ids_sorted = sorted(ids)
33 print("Sorted IDs:", ids_sorted)
34 print("Binary Search Result:", binary_search(ids_sorted, 150))
```

AI LAB END EXAM

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\ai_coding> & C:\Python313\python.exe d:/ai_coding/AA.py
Could not find platform independent libraries <prefix>
Linear Search Result: 2
Sorted IDs: [101, 150, 203, 215, 330]
Binary Search Result: 1
PS D:\ai_coding> █
```

Observation :

In this task, the linear search method was replaced with a more efficient binary search. Linear search checks each element one by one, making it slow for large student ID lists. Binary search, however, works on a sorted list and divides the search range into halves, allowing much faster lookup. The optimized version uses fewer comparisons and reduces the time complexity from $O(n)$ to $O(\log n)$. This shows how choosing a better algorithm can greatly improve performance in student record systems.

Task2:Time Complexity Comparison

Search Method	Best Case	Worst Case	Requirement
Linear Search	$O(1)$	$O(n)$	Works on unsorted list
Binary Search	$O(1)$	$O(\log n)$	Requires sorted list

Observation

- **Binary search is MUCH faster**, especially as n becomes large.
- LMS student record systems with thousands of IDs must use **binary search** for improved performance.